



Bitwise Operators & Logic Gates

Complete Crash Course Notes from Scratch



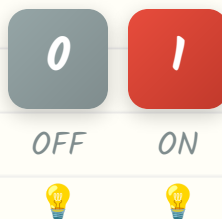
1. What are Binary Numbers?



Binary Number System is a base-2 number system that uses only **two** digits: **0** and **1**. Every computer, smartphone, and digital device uses binary because electronic circuits can only understand two states: **ON (1)** and **OFF (0)**.

Why Binary? The Simple Reason!

Imagine a light switch in your room:



Key Point: A single binary digit (0 or 1) is called a **BIT**. 8 bits together make a **BYTE**. Everything in your computer — photos, videos, games — is stored as millions of 0s and 1s!

Binary Counting (0 to 15)

Decimal	Binary	Decimal	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111



2. Binary ↔ Decimal Conversions

2.1 Decimal to Binary Conversion

Method: Divide by 2 repeatedly and collect remainders from bottom to top!



Example: Convert 13 to Binary

Step 1: $13 \div 2 = 6$ remainder **1**

Step 2: $6 \div 2 = 3$ remainder **0**

Step 3: $3 \div 2 = 1$ remainder **1**

Step 4: $1 \div 2 = 0$ remainder **1**

Answer: Read remainders from bottom $\rightarrow$ top: **1101**

2.2 Binary to Decimal Conversion

Method: Multiply each bit by 2^{position} (from right, starting at 0) and add them up!

 Example: Convert 1011 to Decimal

Position	3	2	1	0
Bit	1	0	1	1
Value	1×2^3	0×2^2	1×2^1	1×2^0
=	8	0	2	1

$$8 + 0 + 2 + 1 = 11$$

So, $1011_2 = 11_{10}$



Real Life: Why does this matter?



Your game score of **255** is stored as **11111111** in 8-bit systems. That's why old games had max scores of 255 — they ran out of bits!



3. Logic Gates - The Building Blocks

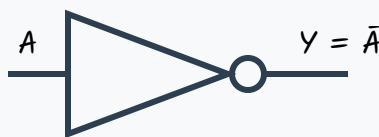


Logic Gates are electronic circuits that perform **logical operations** on one or more binary inputs to produce a single binary output. They are built using **transistors** — tiny electronic switches!



NOT Gate (Inverter)

Function: Outputs the **opposite** of the input. If input is 1, output is 0, and vice versa.



Truth Table

Input (A)	Output (Y = NOT A)
0	1
1	0

0

1

1

0

Transistor Implementation

Input A	Transistor State	Output Y
0 (0V)	OFF (Open)	1 (Connected to Vcc)
1 (5V)	ON (Closed)	0 (Connected to GND)



Real Life Example

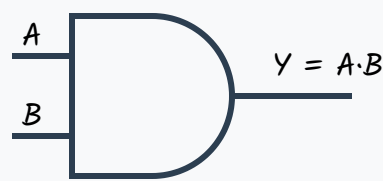


A light switch with a NOT gate: When the switch is OFF (0), the light is ON (1). When you flip the switch ON (1), the light turns OFF (0). It's like a "reverse switch"!



AND Gate

Function: Output is 1 ONLY when ALL inputs are 1. Like saying "Both conditions must be true!"



Truth Table

A	B	Y = A AND B
0	0	0
0	1	0
1	0	0
1	1	1 ✓

Transistor Implementation (2 transistors in series)

Transistor A	Transistor B	Output Y
OFF	OFF	0
OFF	ON	0
ON	OFF	0
ON	ON	1 (Path complete!)

Real Life Example

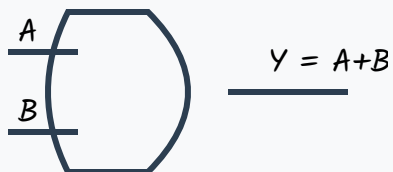


Bank Vault: A vault needs **both a key AND a fingerprint** to open. If you have only the key (1,0) or only the fingerprint (0,1), the vault stays locked. Both must be present (1,1) to unlock!



OR Gate

Function: Output is 1 when **AT LEAST ONE** input is 1. Like saying "Either one or both can be true!"



Truth Table

A	B	Y = A OR B
0	0	0
0	1	1 ✓
1	0	1 ✓

/

/

/ ✓

Transistor Implementation (2 transistors in parallel)

Transistor A	Transistor B	Output Y
OFF	OFF	0
OFF	ON	1 (Path through B)
ON	OFF	1 (Path through A)
ON	ON	1 (Both paths open!)



Real Life Example



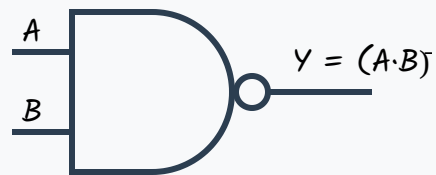
Fire Alarm System: A fire alarm triggers if **smoke is detected OR heat is detected**. Even if only one sensor activates, the alarm rings. You don't need both — either one is enough to save lives!



NAND Gate (NOT + AND)

Function: Output is **opposite of AND**. It's 0 **ONLY** when both inputs are 1. Otherwise, it's 1.

★ **NAND is called a "Universal Gate" — you can build ANY other gate using only NAND gates!**



Truth Table

A	B	AND	Y = NAND
0	0	0	1 ✓
0	1	0	1 ✓
1	0	0	1 ✓
1	1	1	0

Real Life Example



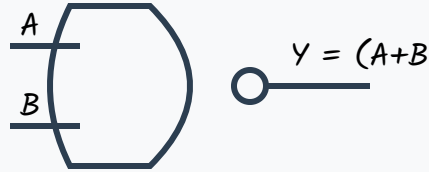
Car Safety: Your car's airbag should NOT deploy if **both** the seatbelt is fastened **AND** the crash sensor is triggered in a minor bump. NAND logic helps prevent false deployments — it stays off (0) only when both conditions are truly met (1,1).



NOR Gate (NOT + OR)

Function: Output is **opposite of OR**. It's 1 **ONLY** when both inputs are 0. Otherwise, it's 0.

★ **NOR is also a "Universal Gate" — just like NAND!**



Truth Table

A	B	OR	Y = NOR
0	0	0	1 ✓
0	1	1	0
1	0	1	0
1	1	1	0

Real Life Example



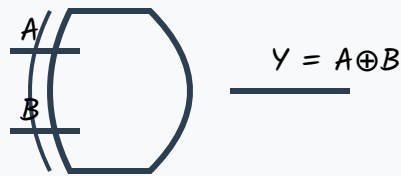
Home Security: A silent alarm triggers **ONLY** when **NO** motion is detected **AND** the door is closed (both 0). If either a burglar moves (1) or the door opens (1), the alarm stops being silent — the NOR output becomes 0, triggering the loud alarm!



EX-OR Gate (Exclusive OR)

Function: Output is 1 when inputs are **DIFFERENT**. It's like asking "Are they **NOT** the same?"

Mnemonic: "EX-OR = EXclusive = Only ONE can win!"



Truth Table

A	B	$Y = A \text{ XOR } B$
0	0	0 (Same)
0	1	1 ✓ (Different!)
1	0	1 ✓ (Different!)
1	1	0 (Same)



Real Life Example



Two-Way Light Switch: In your staircase, two switches control one light. Flipping **either** switch changes the light's

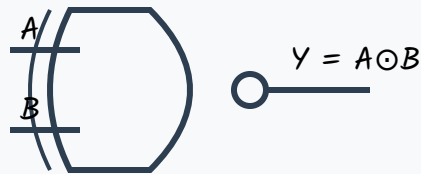
state. If both switches are in the same position, the light is off. If they're different, the light is on. That's EX-OR!



EX-NOR Gate (Exclusive NOR)

Function: Output is 1 when inputs are the SAME. It's the exact opposite of EX-OR!

Mnemonic: "EX-NOR = Equality Detector = Are they the SAME?"



Truth Table

A	B	$Y = A \text{ XNOR } B$
0	0	1 ✓ (Same!)
0	1	0 (Different)
1	0	0 (Different)
1	1	1 ✓ (Same!)

Real Life Example



Password Checker: When you type your password, the computer compares each character. If **your input matches the stored password** (both same = 1,1 or both different positions = 0,0), EX-NOR outputs 1 (Access Granted!). If even one character differs, output is 0 (Access Denied!).

4. Quick Reference - All Gates at a Glance

Gate	Symbol	Expression	Output = 1 when...
NOT	Triangle + ○	$Y = \bar{A}$	Input is 0
AND	D-shape	$Y = A \cdot B$	Both inputs are 1
OR	Curved shape	$Y = A + B$	At least one input is 1
NAND	AND + ○	$Y = (A \cdot B)'$	NOT both 1 (Universal!)
NOR	OR + ○	$Y = (A + B)'$	Both inputs are 0 (Universal!)
EX-OR	OR + extra curve	$Y = A \oplus B$	Inputs are DIFFERENT
EX-NOR	EX-OR + ○	$Y = A \odot B$	Inputs are SAME



5. Practice Problems



Problem 1: Decimal to Binary

Convert **45** to binary.

Show Solution



Problem 2: Binary to Decimal

Convert **110101** to decimal.

Show Solution



Problem 3: Logic Gate Challenge

If $A = 1$, $B = 0$, $C = 1$, find the output of: **(A AND B) OR (B NAND C)**

Show Solution



Problem 4: EX-OR Application

A light is controlled by two switches. Currently both are OFF (0,0) and the light is OFF. You flip switch A to ON. What happens? What if you then flip switch B to ON too?

Show Solution



Problem 5: Universal Gate

Can you make a NOT gate using only NAND gates? How many NAND gates do you need?

Show Solution



6. Bitwise Operators in Programming



Bitwise operators work directly on the binary representation of numbers. They are super fast because computers work in binary natively!

Operator	Symbol	Description	Example (5 & 3)
AND	&	Bitwise AND	101 & 011 = 001 (1)
OR		Bitwise OR	101 011 = 111 (7)
XOR	^	Bitwise XOR	101 ^ 011 = 110 (6)
NOT	~	Bitwise complement	~5 = ...11111010
Left Shift	<<	Shift bits left	5 << 1 = 1010 (10)
Right Shift	>>	Shift bits right	5 >> 1 = 10 (2)



Programming Example

Checking if a number is even or odd:

```
if (num & 1) { /* It's ODD! */ }
```

Why? Because all odd numbers have their last bit as 1. AND with 1 keeps only the last bit!



Real Life: Bitwise in Action



Game Permissions: In online games, permissions are stored as bits:

Bit 0 = Can Chat, Bit 1 = Can Build, Bit 2 = Can Moderate

A player with permission **5** (binary **101**) can Chat and Moderate, but NOT Build!



Made with ❤️ for Learning | Bitwise Operators Complete Guide

Remember: Computers don't understand words — they understand 0s and 1s!